

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: Kottana Roshini

Roll No.: A24126552263

1. TITLE

Working with Node.js Modules — CommonJS, Built-in, Third-party, and ES6 Modules

2. AIM

To understand and demonstrate the module system in Node.js by creating and using CommonJS modules, built-in (core) modules, third-party modules installed via npm, and ES6 modules using import/export syntax.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what a module is and why modularity is important in Node.js.
- To create and use a custom CommonJS module with `require()` and `module.exports`.
- To use Node.js built-in (core) modules such as `os` and `fs`.
- To install and use a third-party module from npm (lodash).
- To understand and implement ES6 modules using `import` and `export`.
- To compare CommonJS and ES6 module syntax.

4. THEORY

A module in Node.js is a reusable block of code whose functionality can be exported and used in other files. Node.js follows a modular architecture where every file is treated as a separate module.

CommonJS is the default module system used by Node.js. It uses `require()` to import a module and `module.exports` (or `exports`) to export functionality from a file.

Built-in (core) modules are modules that come pre-packaged with Node.js and do not need to be installed separately, such as `fs` (file system), `os` (operating system information), `path`, and `http`.

Third-party modules are packages published on the npm registry by other developers. They are installed using `npm install <package-name>` and then imported using `require()` (or `import` in ES6 module mode).

ES6 modules (ECMAScript modules) are the standardized JavaScript module system that uses the `export` and `import` keywords. In Node.js, ES6 modules are enabled either by using the `.mjs` file extension or by setting `"type": "module"` in `package.json`.

The key difference between CommonJS and ES6 modules is that CommonJS modules are loaded synchronously using `require()`, whereas ES6 modules are loaded using the static `import` statement and support features like named exports, default exports, and tree-shaking.

5. ALGORITHM / PROCEDURE

- Create a Node.js project folder for each type of module demonstration.
- For the CommonJS demo, create a module file that defines functions and exports them using `module.exports`.

- Create a separate file that imports the custom module using `require()` and calls its functions.
- For the built-in module demo, import the `os` and `fs` core modules without installing anything.
- Use the `os` module to retrieve system information and the `fs` module to write and read a file.
- For the third-party module demo, run `npm init` and `npm install lodash` to add the package to the project.
- Import `lodash` using `require()` and use its utility functions on sample data.
- For the ES6 module demo, set `"type": "module"` in `package.json` (or use `.mjs` files).
- Create a module file using the `export` keyword and a consumer file using the `import` keyword.
- Run each program using Node.js and record the output.
- Verify the actual output against the expected output for each test case.

6. PROGRAM

6.1 CommonJS Custom Module

`mathUtils.js`

```
// mathUtils.js - Custom CommonJS Module
function add(a, b) {
    return a + b;
}

function subtract(a, b) {
    return a - b;
}

function multiply(a, b) {
    return a * b;
}

module.exports = { add, subtract, multiply };
```

`app.js`

```
// app.js - Using the custom CommonJS module
const mathUtils = require("./mathUtils");

console.log("---- CommonJS Custom Module Demo ----");
console.log("Addition (5 + 3):", mathUtils.add(5, 3));
console.log("Subtraction (5 - 3):", mathUtils.subtract(5, 3));
console.log("Multiplication (5 * 3):", mathUtils.multiply(5, 3));
```

6.2 Built-in (Core) Module

`systemInfo.js`

```
// systemInfo.js - Using Node.js Built-in Modules
const os = require("os");
const fs = require("fs");

console.log("---- Built-in Module Demo (os, fs) ----");
console.log("Operating System Platform:", os.platform());
console.log("CPU Architecture:", os.arch());
console.log("Total Memory (MB):", (os.totalmem() / (1024 * 1024)).toFixed(2));
console.log("Free Memory (MB):", (os.freemem() / (1024 * 1024)).toFixed(2));
console.log("Number of CPU Cores:", os.cpus().length);

// Using fs module to write and read a file
fs.writeFileSync("sample.txt", "Hello from the fs built-in module!");
```

```
const data = fs.readFileSync("sample.txt", "utf8");
console.log("File Content Read Using fs Module:", data);
```

6.3 Third-party Module (lodash)

lodashDemo.js

```
// lodashDemo.js - Using a Third-party Module (lodash)
const _ = require("lodash");

console.log("---- Third-party Module Demo (lodash) ----");

const numbers = [10, 20, 30, 40, 50];
console.log("Original Array:", numbers);
console.log("Chunked Array (size 2):", _.chunk(numbers, 2));
console.log("Reversed Array:", _.reverse([...numbers]));
console.log("Maximum Value:", _.max(numbers));
console.log("Sum of Array:", _.sum(numbers));

const student = { name: "Kottana Roshini", rollno: "A24126552263" };
console.log("Cloned Object:", _.cloneDeep(student));
```

6.4 ES6 Module (import / export)

package.json

```
{
  "name": "es6-module-demo",
  "version": "1.0.0",
  "type": "module"
}
```

mathES6.mjs

```
// mathES6.mjs - Custom ES6 Module
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}

export default function multiply(a, b) {
  return a * b;
}
```

appES6.mjs

```
// appES6.mjs - Using ES6 import/export
import multiply, { add, subtract } from "./mathES6.mjs";

console.log("---- ES6 Module Demo (import/export) ----");
console.log("Addition (7 + 4):", add(7, 4));
console.log("Subtraction (7 - 4):", subtract(7, 4));
console.log("Multiplication (7 * 4):", multiply(7, 4));
```

7. CODE EXPLANATION

Code	Explanation
module.exports	Exposes functions/values from a file so they can be used in other files.
require("./mathUtils")	Imports a custom CommonJS module using its relative file path.
require("os") / require("fs")	Imports Node.js built-in modules for system info and file handling.
os.platform() / os.cpus()	Returns operating system and CPU details of the host machine.
fs.writeFileSync() / fs.readFileSync()	Writes to and reads from a file synchronously.
npm install lodash	Downloads and adds the lodash third-party package to node_modules.
require("lodash")	Imports the third-party lodash module for use in the program.
"type": "module"	Tells Node.js to treat .js files in the project as ES6 modules.
export / export default	Exposes named or default values from an ES6 module.
import ... from "./file.mjs"	Imports named and default exports from an ES6 module.

8. TEST CASES AND EXECUTION DATA

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Run app.js (CommonJS)	Correct sum, difference and product printed	Correct sum, difference and product printed	Pass
TC-02	Run systemInfo.js (built-in modules)	OS, CPU and memory details printed; file written and read	OS, CPU and memory details printed; file written and read	Pass
TC-03	Run lodashDemo.js (third-party module)	Array and object operations performed using lodash	Array and object operations performed using lodash	Pass
TC-04	Run appES6.mjs (ES6 module)	Correct sum, difference and product printed using import/export	Correct sum, difference and product printed using import/export	Pass

9. OUTPUT

Each program was executed in the terminal using the Node.js runtime. The screenshots below show the actual output produced for every module type.

Output 1: CommonJS Custom Module (app.js)

```
Terminal — CommonJS Module

$ node app.js
---- CommonJS Custom Module Demo ----
Addition (5 + 3): 8
Subtraction (5 - 3): 2
Multiplication (5 * 3): 15
```

Output 2: Built-in Module — os and fs (systemInfo.js)

```
Terminal — Built-in Module

$ node systemInfo.js
---- Built-in Module Demo (os, fs) ----
Operating System Platform: linux
CPU Architecture: x64
Total Memory (MB): 3997.87
Free Memory (MB): 3727.81
Number of CPU Cores: 1
File Content Read Using fs Module: Hello from the fs built-in module!
```

Output 3: Third-party Module — lodash (lodashDemo.js)

```
Terminal — Third-party Module

$ node lodashDemo.js
---- Third-party Module Demo (lodash) ----
Original Array: [ 10, 20, 30, 40, 50 ]
Chunked Array (size 2): [ [ 10, 20 ], [ 30, 40 ], [ 50 ] ]
Reversed Array: [ 50, 40, 30, 20, 10 ]
Maximum Value: 50
Sum of Array: 150
Cloned Object: { name: "Kottana Roshini", rollno: "A24126552263" }
```

Output 4: ES6 Module — import/export (appES6.mjs)

```
Terminal — ES6 Module

$ node appES6.mjs
---- ES6 Module Demo (import/export) ----
Addition (7 + 4): 11
Subtraction (7 - 4): 3
Multiplication (7 * 4): 28
```

10. RESULT

Thus, the module system in Node.js was successfully studied and demonstrated. A custom CommonJS module was created and used with `require()/module.exports`, built-in modules (`os`, `fs`) were used to read system information and handle files, a third-party module (`lodash`) was installed via `npm` and used for array and object operations, and an ES6 module was implemented using `export` and `import` syntax. All programs were executed and verified successfully.